Natural Language Processing (NLP)

Text Features & Classification

Classical Text Features & Classification

By:

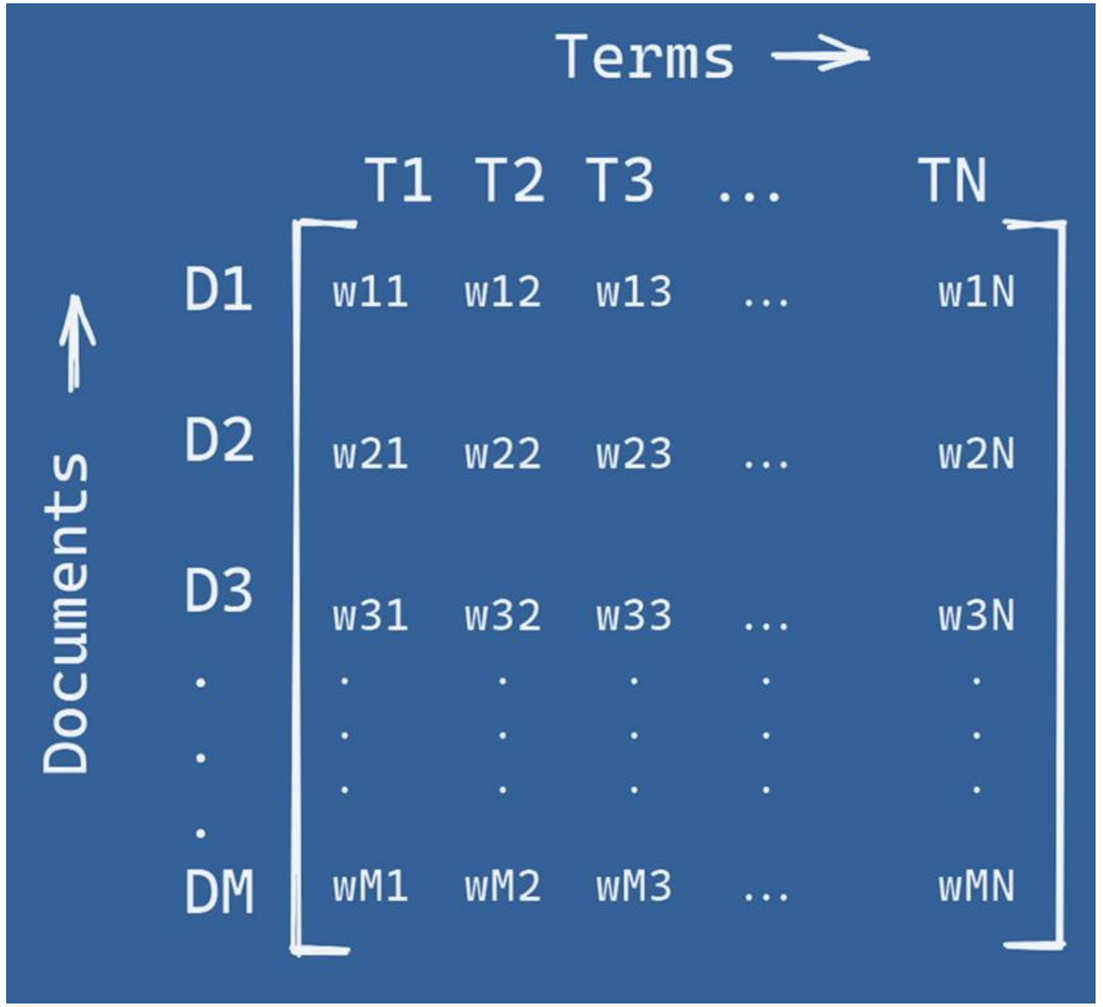
Dr. Zohair Ahmed

www.youtube.com/@ZohairAI Subscribe

www.begindiscovery.com

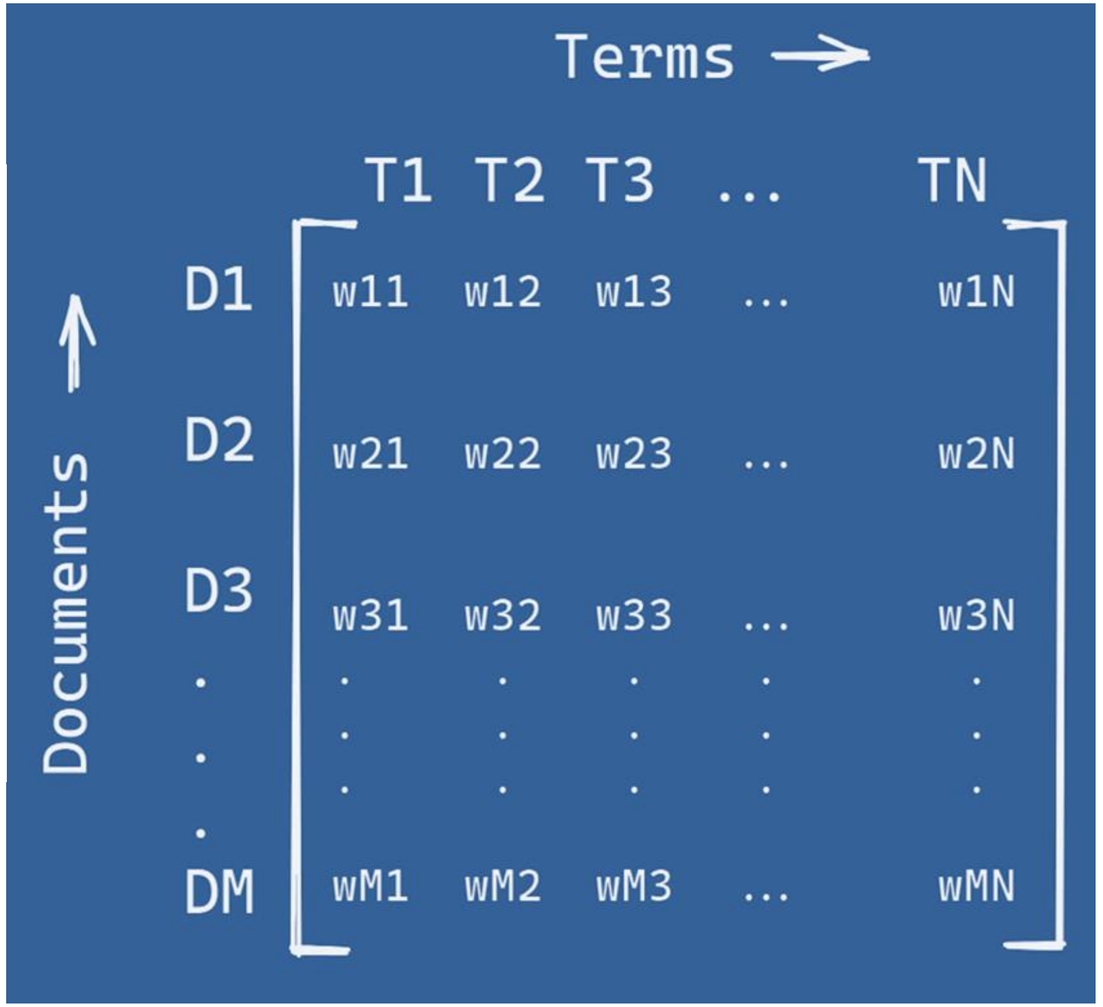
Bag-of-Words (BoW)

- **Bag-of-Words** represents each document as a multiset (“bag”) of its tokens, ignoring order. You build:
- A **vocabulary**: all unique tokens in the corpus.
- A **document–term matrix**: rows = documents, columns = vocabulary terms, entries = some numeric value per term in that document.
- No word order, syntax, or semantics are encoded, only how often each token appears.



Bag-of-Words (BoW)

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0



Vocabulary, Document–Term Matrix, Sparsity

- The vocabulary is the set: $V = \{w_1, w_2, \ldots, w_{|V|}\}$ of all distinct tokens after preprocessing (e.g., tokenization, lowercasing, maybe stopword removal).
- **Definition (Document–Term Matrix)**
  For N documents and vocabulary size |V|, define a matrix: $X \in \mathbb{R}^{N \times |V|}$
- N = number of documents, |V|= size of your vocabulary
- $X_{\{ij\}}$= information about term w_j in document d_i
- Row i: feature vector for document d_i.
- Column j: feature across all documents for term w_j.
- **Sparsity**
- Most entries in X are **zero** because any single document uses only a small fraction of the full vocabulary.
- **Sparse matrix**: a matrix where the proportion of zero entries is very high.
- In practice, |V| can be $10^4 to\ 10^6$, while each document might use only tens or hundreds of those terms.

Binary vs Counts vs Normalized Frequencies

- Let $c_{\{ij\}}$be the raw count of term w_j in document d_i.
- **Binary representation**
- $X_{ij} = \begin{cases} 1 & \text{if } c_{ij} > 0 \\ 0 & \text{otherwise} \end{cases}$
  - Captures **presence/absence** only.
  - Good when *any* occurrence of a word is already very informative (e.g., sentiment words like “awful”, domain keywords).
- **Raw counts**
- $X_{ij} = c_{ij}$
  - Captures how many times a word appears.
  - Assumes **more frequent = more important** for that document.
- **Normalized frequencies**
- Common: **term frequency** normalized by document length:
- $X_{ij} = \frac{c_{ij}}{\sum_{k=1}^{|V|} c_{ik}}$
  - This is the **relative frequency** of term $w_j$in document d_i.
  - Helpful because documents can have very different lengths; normalization makes features comparable across documents.
- Binary: “Did the word occur?”
- Counts: “How many times did it occur?”
- Normalized: “How big a share of the document is this word?”

Binary vs Counts vs Normalized Frequencies

Small Example

- Document d_1:
  - awesome awesome movie
- Vocabulary: {awesome, movie}
- Counts:
  - awesome: 2
  - movie: 1
  - total words = 3
- **Binary:**
  - awesome → 1
  - movie → 1
- **Raw counts:**
  - awesome → 2
  - movie → 1
- **Normalized frequency:**
  - $awesome = \frac{2}{3}, movie = \frac{1}{3}$

Binary vs Counts vs Normalized Frequencies

Small Example

- Document d_1:
  - awesome awesome movie
- Vocabulary: {awesome, movie}
- Counts:
  - awesome: 2
  - movie: 1
  - total words = 3
- **Binary:**
  - awesome → 1
  - movie → 1
- **Raw counts:**
  - awesome → 2
  - movie → 1
- **Normalized frequency:**
  - $awesome = \frac{2}{3}, movie = \frac{1}{3}$

TF-IDF

Term Frequency (TF)

For a term t in document d:

- $\text{tf}(t, d) = \frac{\textit{count of } t \textit{ in } d}{\textit{total tokens in } d}$
- You can also use raw counts as TF; the normalized version is more common.

Document Frequency (DF)

- For a term t:
- $\text{df}(t) = \textit{number of documents where } t \textit{ appears}$
- **Inverse Document Frequency (IDF)**

For corpus with N documents:

- $\text{idf}(t) = \log\left(\frac{N}{df(t)}\right)$
- Often with smoothing to avoid division by zero:
- $\text{idf}(t) = \log\left(\frac{N+1}{df(t)+1}\right) + 1$
- **TF-IDF weight**
- For term t in document d:
- $\text{tfidf}(t, d) = tf(t, d) \cdot idf(t)$

Intuition: Upweight Rare but Informative Terms

- High **TF**: the word is important *within* that document.
- High **IDF**: the word is **rare across documents**, so it carries more discriminative information.
- Words like “the”, “is”:
- High DF, IDF ≈ 0 → low TF-IDF even if frequent.
- Words like “neural”, “bankruptcy”, “excellent”:
- Lower DF, higher IDF → higher TF-IDF when they appear.
- So, TF-IDF:
- **Down weights** very common, uninformative words.
- **Upweights** rare but potentially informative words.

BoW vs TF-IDF

- **BoW (binary or counts)**
- Pros:
  - Simple.
  - Often works surprisingly well for basic classification tasks.
- Cons:
  - Treats all terms equally regardless of how common they are in the corpus.
  - Common function words can dominate similarity and classification if not handled.
- **TF-IDF**
- Pros:
  - Incorporates **global corpus statistics** (IDF).
  - Better for tasks like information retrieval or document similarity where we care about distinctive words.
- Cons:
  - Still ignores word order and semantics.
  - Slightly more complex to compute (need global DF statistics).
- “BoW with counts = purely local view (within one document).”
- “TF-IDF = local importance (TF) * global rarity (IDF).”

BoW vs TF-IDF

- **BoW (binary or counts)**
- Pros:
  - Simple.
  - Often works surprisingly well for basic classification tasks.
- Cons:
  - Treats all terms equally regardless of how common they are in the corpus.
  - Common function words can dominate similarity and classification if not handled.
- **TF-IDF**
- Pros:
  - Incorporates **global corpus statistics** (IDF).
  - Better for tasks like information retrieval or document similarity where we care about distinctive words.
- Cons:
  - Still ignores word order and semantics.
  - Slightly more complex to compute (need global DF statistics).
- “BoW with counts = purely local view (within one document).”
- “TF-IDF = local importance (TF) * global rarity (IDF).”

Classification Models for Text

- **Discriminative =** draws the best boundary between classes
- **Generative =** tries to understand how each class generates the data
- **Logistic Regression = Discriminative**
- A **discriminative** model learns: $P(y \mid x)$
  Given this input x, what is the probability that the class is y?
- It does **NOT** try to model how x (the features) are produced.
- **Logistic Regression ONLY learns the decision boundary.**

- It directly estimates:
- $P(y = 1 \mid x) = \sigma(w^T x)$
- It asks:
- "What's the probability this is class 1 based on the features?"
- It does **not** model:
  - how words appear in spam emails
  - how pixels form a "cat" image
  - how features are generated
  - It simply tries to **separate classes**.

Classification Models for Text

- **Naive Bayes = Generative**
- A **generative** model learns how data for each class is **generated**.
- It models: $P(x \mid y)$and$P(y)$
- Using these, it computes:$P(y \mid x)$
- So yes, it **also produces probabilities**,
- BUT: **it first tries to understand the distribution of features for each class.**
- **Naive Bayes tries to learn how each class "produces" its features.**

For example, in spam classification:

- For each class (spam / not spam), it estimates:
- Probability of seeing the word "offer"
- Probability of seeing "money"
- Probability of seeing "meeting"
- Probability of seeing "free"
- etc.
- It learns two models:
- How spam emails are generated
- How non-spam emails are generated
- Then compares:
- "Given this email, which class is more likely to generate it?"

Decision Boundary

- The **decision rule**: predict class 1 if:
- $P(y = 1 \mid x) = \sigma(w^\top x + b) \geq 0.5$
- **Equivalently, since:** $\sigma(z) \geq 0.5$,

$$w^\top x + b \geq 0$$

- So logistic regression learns a **linear decision boundary** in feature space:
  - A hyperplane separating class 0 from class 1.

Cross-Entropy Loss for Classification

- To train logistic regression we minimize the **cross-entropy (logistic) loss** over the training set.
- Used to train **logistic regression** for **binary classification**.
- The model outputs: $\hat{y} = \sigma(w^\top x + b)$
- Where $\hat{y}$= predicted probability of class **1**
- $y \in \{0,1\}$= true label
- **Loss for ONE training example**
- $L^{(i)} = -[y^{(i)}\log(\hat{y}^{(i)}) \ + \ (1 - y^{(i)})\log(1 - \hat{y}^{(i)})]$

- This formula simply means:
- **If the true label is 1:**
- $L = -\log(\hat{y})$
- If model predicts **0.99** → loss is tiny
- If model predicts **0.01** → loss is huge
- **If the true label is 0:**
- $L = -\log(1 - \hat{y})$
- If model predicts **correct 0** → loss small
- If model predicts **wrong 1** → loss huge

Cross-Entropy Loss for Classification

- **Loss for the whole dataset**
- For **m examples**:
- $L = \frac{1}{m} \sum_{i=1}^{m} L^{(i)}$
- Just the **average of all individual losses**.
- The training goal:
- **Adjust $w$and b**
- **So that total loss becomes small**
- **Using gradient descent**

Cross-Entropy Loss for Classification

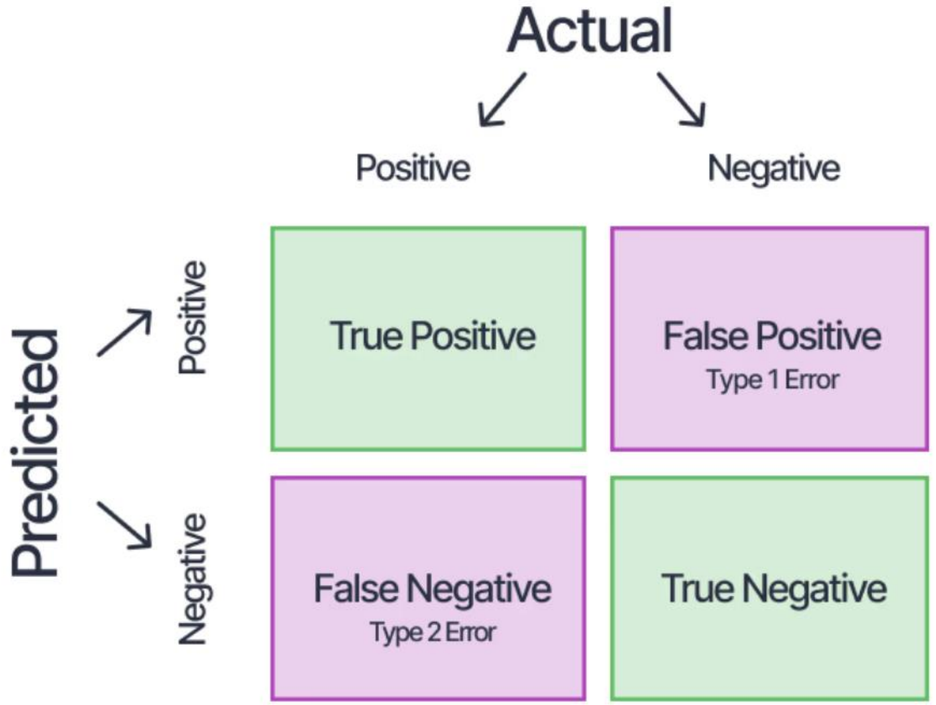
- **Mini Example**
- Suppose:

  True label = 1

  Model predicts: $\hat{y} = 0.9$
- $L = -\log(0.9) = 0.105$
- Small loss $\rightarrow$ model is doing well.
- If model predicts: $\hat{y} = 0.1$
- $L = -\log(0.1) = 2.30$
- Large loss $\rightarrow$ model is doing badly.

Evaluation for Classification

- The idea that **accuracy alone is often not enough**, especially with class imbalance.
- Confusion Matrix: For binary classification (positive vs negative), a confusion matrix is:
  - **TP**: correctly predicted positives.
  - **TN**: correctly predicted negatives.
  - **FP**: predicted positive but actually negative.
  - **FN**: predicted negative but actually positive.



Evaluation for Classification

- **Precision (Positive Predictive Value)**
- Precision $= \frac{TP}{TP+FP}$
- “How many predicted positives are actually correct?”
- **Recall (Sensitivity)**
- Recall $= \frac{TP}{TP+FN}$
- “How many actual positives did we successfully find?”

- **F1 Score**
- Harmonic mean of precision and recall:
- $F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$
- High only when **both** precision and recall are high.
- Useful when classes are imbalanced or when FP/FN costs matter.

Macro vs Micro Averaging (Multiclass / Multilabel)

- For multiple classes (e.g., sentiment: positive, negative, neutral; or topic classes):
- **Macro-Averaged Metrics**
- **Compute precision, recall, and F1 separately for each class**
  Then take the average across classes
- This treats **all classes equally**, even if one class has many more samples than others.
- **Step 1: Compute per-class metrics**
- For each class k:
- Precision: $\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}$
- Recall: $\text{Recall}_k = \frac{TP_k}{TP_k + FN_k}$
- F1 score: $F_{1,k} = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$
- You compute this **for each class** (k = 1, 2, …, K).
  Step 2: Average across classes
- For K classes (e.g., K = 3):
- $\text{MacroF}_1 = \frac{1}{K} \sum_{k=1}^{K} F_{1,k}$
- This is the **correct formula**.

- **Why use macro-average?**
- Treats **all classes equally**
- Good when classes are **imbalanced**
  (small classes matter as much as big ones)

- **Tiny Example (3 Classes)**
- Suppose F1 scores for classes A, B, C are:
- $F_{1,A} = 0.8$
- $F_{1,B} = 0.6$
- $F_{1,C} = 0.4$
- Then:
- $\text{MacroF}_1 = \frac{0.8+0.6+0.4}{3} = \frac{1.8}{3} = 0.6$
- **One-Sentence Summary**
- **Macro-average gives equal weight to every class by averaging per-class metrics.**

ROC Curve (Receiver Operating Characteristic)

- The **ROC curve** plots:
- **True Positive Rate (TPR)**TPR $= \frac{TP}{TP+FN} =$ Recall

  False Positive Rate (FPR) FPR $= \frac{FP}{FP+TN}$
- Each point on the ROC curve corresponds to using a **different classification threshold** on the model's predicted score or probability.
- Example:
- threshold = 0.9
- threshold = 0.7
- threshold = 0.5
- threshold = 0.3

  → each gives different (TPR, FPR)
- ROC shows the trade-off between **catching positives** and **avoiding false alarms**.
- **AUC (Area Under the ROC Curve)**
- AUC is a **single number between 0 and 1**.
- **AUC = probability that the classifier ranks a random positive sample higher than a random negative sample.**
- Examples:
- AUC = **0.5** → random guessing
- AUC = **1.0** → perfect classifier
- AUC = **0.9** → very strong classifier
- **Why ROC/AUC Is Useful**
- **Threshold-independent**

  (unlike accuracy, precision, recall)
- Works well under **class imbalance**

  because it focuses on TPR vs FPR rather than raw counts.
- **ROC curve:** plot of TPR vs FPR across thresholds
- **AUC:** probability the model ranks positives above negatives
- **Good for imbalanced data** and when choosing thresholds

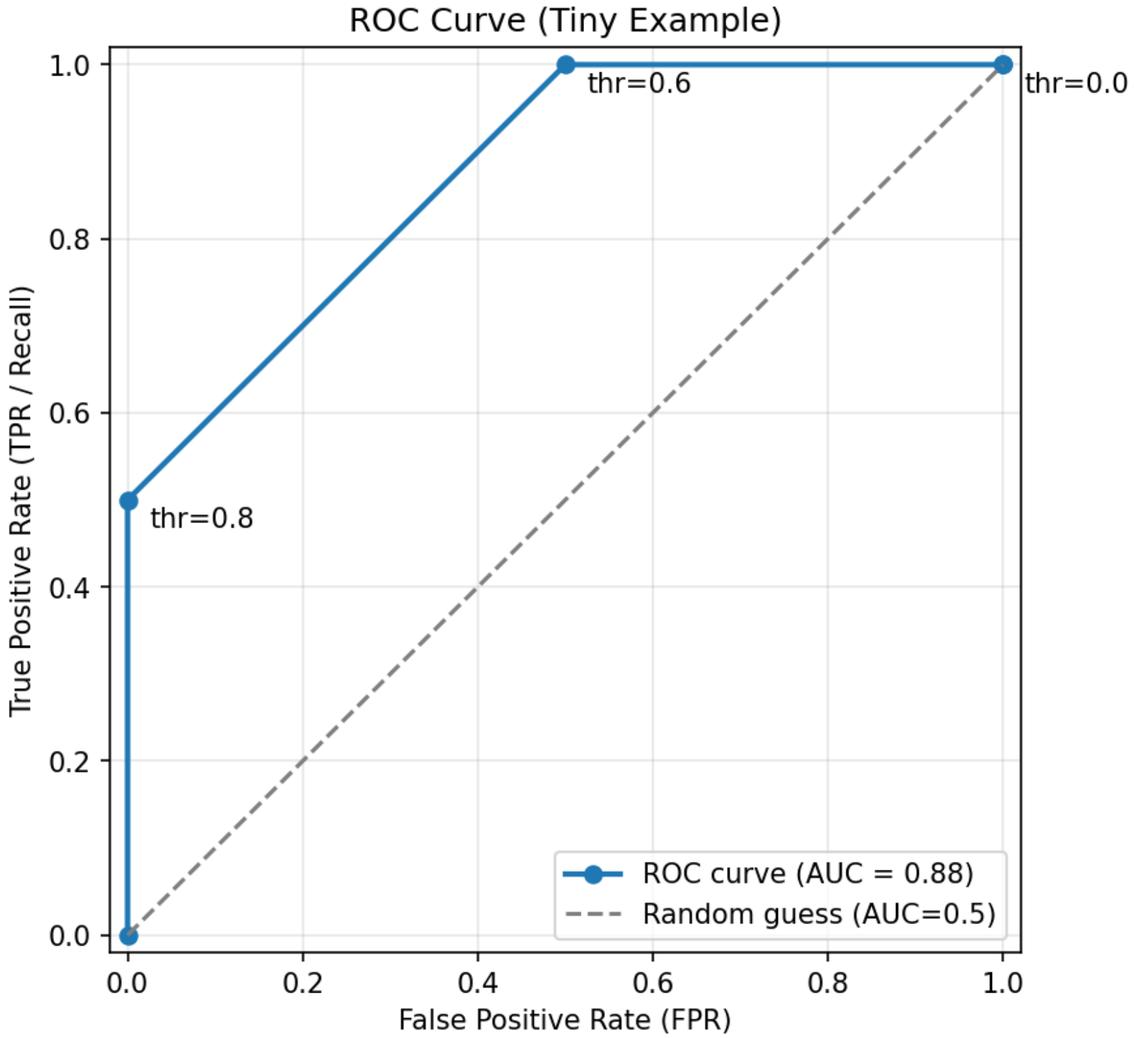
ROC Curve (Receiver Operating Characteristic)

- **Small ROC Example (Binary Classification)**
- Suppose we have **4 samples**:
- Higher score = more likely positive.
- We will sweep thresholds and compute:
- **TPR = TP / (TP + FN)**
- **FPR = FP / (FP + TN)**
- **Threshold = 0.8**
  - Predicted positive: scores ≥ 0.8 → {A}
    Predicted negative → {B, C, D}
  - TP = 1 (A)
  - FN = 1 (B)
  - FP = 0
  - TN = 2 (C, D)
  - $TPR = \frac{1}{2} = 0.5, FPR = 0$
  - Point: **(FPR=0, TPR=0.5)**
- **Threshold = 0.6**
  - Predicted positive → {A, B, C}
    Predicted negative → {D}
  - TP = 2 (A, B)
  - FN = 0
  - FP = 1 (C)
  - TN = 1 (D)
  - $TPR = \frac{2}{2} = 1.0, FPR = \frac{1}{2} = 0.5$
  - Point: **(FPR=0.5, TPR=1.0)**

Sample	True Label	Model Score
A	1	0.9
B	1	0.7
C	0	0.6
D	0	0.2

- **Threshold = 0.0**
  - Predict everything positive.
  - TP = 2
  - FN = 0
  - FP = 2
  - TN = 0
  - $TPR = 1.0, FPR = 1.0$
- Point: **(FPR=1.0, TPR=1.0)**
- **ROC Curve Points**
- In order:
  - **(0, 0)** — threshold so high nothing predicted positive
  - **(0, 0.5)** — threshold = 0.8
  - **(0.5, 1.0)** — threshold = 0.6
  - **(1.0, 1.0)** — threshold = 0
  - These points form the ROC curve.
- **Interpretation**
  - As threshold decreases, more samples are predicted positive
  - TPR increases → good
  - FPR also increases → trade-off
  - ROC curve shows this trade-off clearly
- **One-Sentence Summary**
- **A ROC curve is simply a plot of (FPR, TPR) at different thresholds; AUC measures how well the model ranks positives above negatives.**

ROC Curve (Receiver Operating Characteristic)



Case Study: Sentiment Analysis Pipeline

Step 1: Preprocessing

- **Tokenization**
  - Turn raw text into tokens (word or subword level).
- **Normalization**
  - Lowercasing.
  - Optional: removing punctuation, numbers, or not (discuss trade-offs).
- **Stopword handling**
  - Possibly remove very common function words (but note that some, like “not”, are crucial in sentiment).
- **Optional steps**
  - Stemming or lemmatization.
  - Handling emojis, hashtags, etc., depending on domain (e.g., social media).

Step 2: Feature Extraction (TF-IDF)

- Build a vocabulary from training data.
- Construct a **TF-IDF document–term matrix**:
  - Each review → TF-IDF vector.
- Optionally:
  - Restrict vocabulary to top K frequent terms.
  - Remove very rare terms that appear in fewer than a threshold number of documents.

Step 3: Choose a Classifier

- Two natural choices from this week:
- **Multinomial Naïve Bayes**
  - Fits very fast.
  - Often a strong baseline for text.
- **Logistic Regression**
  - Often achieves higher accuracy given enough data and good regularization.
  - Interpretable feature weights (which words push toward positive vs negative).
- **Training:**
- Split labeled dataset into **train/dev/test** or use cross-validation.
- Fit classifier parameters on training set.
- Tune hyperparameters (e.g., regularization strength) on dev set.

Step 4: Evaluation

- Confusion matrix (on test set).
- Precision, recall, F1 (per class).
- Macro and micro F1 (if you include a neutral class).
- Optionally, ROC/AUC for binary sentiment.

Discuss:

- How many positive reviews are misclassified as negative (and vice versa)?
- Which type of error is more harmful (depends on application).

Step 5: Interpretation and Error Analysis

- For **logistic regression**, each feature weight w_j tells us:
- $If\ w_j > 0$: presence of word $w_j$pushes prediction toward the positive class.
- $If\ w_j < 0$: pushes toward the negative class.
- $Magnitude\ |w_j|$: how strong the effect is.

Classroom exercise :

- Show the top 10 most positive and most negative words according to model weights.
- Show misclassified examples and ask:
  - Which words might be misleading the model?
  - Are there cases where **negation** (“not good”) fools the BoW model because it ignores word order?